

Question #2: Analytical Reasoning

Pose Estimation for Padel Hits

Alejandro Campayo

September 23, 2026

Q1. Root causes of validation-to-new-club performance drop

Strong performance on validation and a failure in a new club means that the difference between **validation and training data** is smaller than the difference between **training and real deployment**: there is a **domain shift**. What worries me most is that the validation metrics do not show it, which points to **data leakage**: if validation was recorded in the same club or with the same camera angles as training, the model has **overfitted to one particular setup**.

To isolate the problem:

- **Training vs. validation.** They should be **two genuinely independent subsets**. If they are not, validation loss tracks training loss very closely and we cannot tell when overfitting happens or whether the run is healthy.
- **Validation vs. test.** Look for what changes between them: **scale, perspective, FoV, resolution, color of the ball / field**, etc.
- **Probing.** A cheap check: take the features extracted by the trained model and train a **tiny classifier** to predict what I suspect is breaking it (the padel field, the camera resolution, the perspective). If that classifier is accurate, the representation **encodes this feature** and the decisions may be affected by it.
- **Annotate a few frames from the failing club.** The system is a pipeline, so the most direct test is to label a few hundred frames there and measure the two stages separately: first the pose estimation on its own, and then the hit detection fed with those ground-truth keypoints instead of the predicted ones. If the pose is already wrong, the problem is in the perception stage. If the pose is fine and the hits are still wrong, the problem is in what consumes it.

Q2. Impact of inconsistent annotations and label-quality strategy

Impact on the model

If annotators label differently, the model has **no way of knowing** whether the elbow should go slightly higher or lower: it depends only on the annotator, and we give the model no information about **who labelled what**. To minimise the loss it will predict what the **average annotator** considers correct, but it can also pick up **spurious correlations** that make predictions unreliable. The same happens with the **timestamps**: label quality has a **direct impact** on performance, and with random noise as labels the model would learn nothing useful. **Hit classification** is affected too, since the hit type is probably inferred from wrist and elbow relative to the racket at the moment of contact, and how much will depend on the **architectural choices**. Disagreement between annotators also sets a **noise floor**: the model cannot be more consistent

than the people who labelled the data, so measuring how much they disagree tells us when it stops being worth improving the model and we should improve the labels instead.

How to detect it, and what to do

Everything depends on whether we **know who annotated what** (for example, one folder or file naming per annotator).

- **If we know it**, we can look for systematic differences, e.g. the **average forearm length**. An annotator who places the wrist in the middle of the hand gives a consistent offset in every clip, so we can not only detect the bias but **correct it**, shortening the elbow-to-wrist vector by that amount. Same for timestamps: if one annotator always marks the hit with the ball at distance x from the racket and another at distance y , that is again a correctable bias.
- **If we do not know it**, it gets more complicated. Then the decision depends on how **bad** the annotations are, how **expensive** relabelling would be, and how much performance drops if we simply **filter those parts out** (an **ablation run** makes sense here).

Reweighting adds complexity and possible **over-engineering** (you have to choose a weight per partition), so I would not go there until everything else is exhausted. I would also watch **sub-metrics for smashes** and check whether (1) overall precision and (2) smash precision improve or get worse.

Q3. Handling rare but important smash events

First, without retraining

The cheapest thing is to experiment with the **decision threshold**: lowering it raises recall on rare cases **with no retraining**, and the model may already know more about smashes than the **argmax** suggests. The trade off is **more recall, less precision** on smashes. If this is not enough, we retrain and modify the dataset or the loss.

The dataset

If smashes are very rare, the model can reach a very good training loss **without ever fitting them**. Augmenting them makes the model see these cases more often and **forces it to fit this hit type**:

- **Oversampling** (repeating clips): no new information, only more gradient weight, and it multiplies the risk of **memorising those clips**.
- **Augmentation**: mirroring, cropping / scaling, adding noise, etc.
- **Acquiring real smashes**: this is the **best solution**, since it is the only one adding **real new information** (and it lets me go play padel). A cheap way to get them from footage we already have is to run the current model with a **deliberately low smash threshold**, so it over-predicts smashes, and send only those candidate windows to be checked and annotated. The annotators then spend their time on smashes instead of on the footage that contains none. If we do not have any extra footage: book a padel court, practise smashes with friends, record them and add them to the dataset. I offer myself as a volunteer for the job :).

We can also update the **loss** so that a missed smash is **penalised more heavily**.

Evaluation

Here I would be **more careful**, because any change makes the results **not comparable** with other versions of the model.

- If validation already has **enough smashes** and they are correctly labelled, **leave it as it is**.
- If there are too few, the conclusions may not be **statistically significant**. In that case I would add smashes to validation as well, but only by **acquiring real ones**, never by oversampling or augmentation. Loss results would not be comparable anymore.
- Keeping evaluation **untouched** is what lets me tell whether the training changes are real or just **overfitting artifacts**: the same augmentation on both sides could make me believe the model improved when it is only overfitting.

Trade-offs

Weighting smashes too heavily will probably **degrade the other hit types**, so how much to weight them depends on **how important they are for the product**. Augmentation costs **training time** and changing the loss can add **instability**. If training time is not a bottleneck, I would go with **augmenting the dataset**.

Q4. Stable-but-less-accurate vs. accurate-but-jittery model

If two runs give noticeably different results, I would first train **more models with different seeds**, check that they **converged properly** and understand what randomness produced this.

For production I would **ship the stable one**: it is easier to justify a model with lower precision than one that from time to time gives a completely **unpredictable response**. It depends of course on how much precision we lose and how often the nonsense happens, but for a product a **predictable model is better**, because you have to know what you are selling and be able to speak for it when it makes mistakes.

For development I am **more interested in the jittery model**:

- Which scenarios **break** it, and when exactly it produces inconsistent timing or jittery keypoints.
- Are those jitters **justified**? If they are (the ball is not visible, it is occluded, or similar), that would **change my decision** between the two models.
- How do we even measure the jitter? **Per-frame keypoint error cannot see it**, because it compares every frame with its ground truth independently, so a model whose predictions oscillate between consecutive frames can still look better on average. To see it we have to compare each frame **with the previous timesteps**: displacement and jerk between consecutive frames, especially in segments where the player is almost still and the real movement should be close to zero.
- Can they be removed in **postprocessing**? A person moves smoothly and cannot be “teleported”, so keypoints that jump can be identified as **noise** and filtered out.

Q5. Proposed model for this application

I would split the architecture into **four different problems**:

1. **Pose estimation of the players.** An **off-the-shelf model** ([RTMO](#) or [YOLO26-pose](#)), evaluated to see how it performs and whether **fine-tuning** is needed, plus lightweight **multi-person tracking** so keypoints carry a stable identity. I would lean towards RTMO because it is **one-stage**: it finds all the keypoints in the image in a single forward pass, so its cost does not depend on how many people are in the frame. A classic **top-down** model works the other way around, because it first detects each person and then runs a separate pose network on every crop. With four players on a padel court that means running the pose network four times per frame.
2. **Ball tracking.** It probably needs a **higher frequency** than the pose estimation, or a couple of frames at a time, since the ball is a **tiny object** moving much faster. I would take a model **pretrained on tennis** or a similar sport and fine-tune it for padel.
3. **Racket tracking.** Same approach, pretrained plus fine-tuning. I am **not sure it is necessary**, so I would first evaluate whether the racket can be inferred accurately enough from the **hand position**.
4. **Hit classifier.** Trained on the **outputs of the previous stages** (poses, ball and racket positions), a **tiny ML model** is enough to determine the hit type, and it can be **image agnostic**.

For 1, 2 and 3 I would pick models that run **fast on CPU**, ideally with no GPU at all, which is why I would prefer a **CNN backbone** over a transformer one: vision transformers are **less CPU friendly**. The main issue would probably be **licensing**: checking which models we are allowed to use.